

IAM-95 LAB: Terraform IAM Provisioning — Groups, Policies, Boundaries, Bindings

Series: IAM — IAM Administration | **Reference:** 95 — Terraform IAM Provisioning LAB | **Created:** July 2026 | **Last Updated:** 08/24/2026

Overview

Hands-on lab: provision a complete persona + team IAM model with **Terraform**. The lab is **self-contained** — Step 1 gives you every module and the composition that wires them, so there is nothing to clone. Every command and expected output comes from a real run, verified live against the Account Management API on 07/16/2026, and the HCL is schema-verified against `dynatrace-oss/dynatrace v1.100.0`.

This is one of **three provisioning flavors** — pick the one that fits your workflow:

Flavor	Notebook	Best for
curl / shell	IAM-12	One-off scripts, CI glue
Terraform (this lab)	IAM-95	Declarative IaC, drift detection, team review via plan/apply
Python	IAM-96	Programmatic onboarding, integration into portals/tooling

What you will build (the BPN pattern from IAM-05 / IAM-10):

```
Persona groups (Standard / Power / Admin)
└─ account-level persona policies, bound 1:1

Team groups – driven entirely by ONE `teams` variable
└─ Policy binding (scoped to ONE environment)
    └─ Policy: ONE shared parameterized template ← created once, reused
    by every team
        |
        | ALLOW storage:logs:read WHERE storage:dt.security_context IN
        | ("${bindParam:team}");
    └─ Boundary: one per team (defense-in-depth)
        storage:dt.security_context IN ("&lt;team-context&gt;");
```

Time: 30–45 minutes | **Difficulty:** Intermediate | **Cost:** No consumption impact (IAM objects only)

Table of Contents

1. [Get the Project and Inspect the Modules](#)
2. [Configure Credentials and Teams](#)
3. [Initialize and Plan](#)
4. [Apply](#)
5. [Validate](#)
6. [The Payoff: Onboard a Team with Three Lines](#)
7. [Clean Up](#)
8. [Troubleshooting](#)
9. [Validation Checklist](#)
10. [References](#)

Prerequisites

Requirement	Details
Terraform	>= 1.3 (<code>terraform version</code>)

Requirement	Details
Project	None to fetch — Step 1 builds the whole tree from this notebook
Account UUID	Account Management → Account settings
OAuth client	Account Management → Identity & access management → OAuth clients
OAuth scopes	account-idm-read account-idm-write iam:policies:read iam:policies:write iam:bindings:read iam:bindings:write iam:boundaries:read iam:boundaries:write account-env-read
Prior reading	IAM-04 (policy authoring), IAM-05 (boundary design), IAM-10 (templated policies)

⚠ Safety rails (do not skip):

1. Run this lab only against a **sanctioned test tenant** — confirm the environment ID with your team before executing.
2. Keep `name_prefix` set (e.g. `TFTEST-`) so every created object is unmistakably a lab artifact.
3. Keep `binding_environment_id` set. IAM groups/policies/boundaries are **account-level** objects; the binding is what grants access, and environment-scoping it confines the lab's effect to one environment.

Step 1 — Build the Project

This lab is **self-contained**: every file is below, so there is nothing to clone. Create the tree, then write each file into the path named above it.

```
mkdir -p dt-iam-
terraform/modules/{iam_groups,iam_policies,policy_boundaries,policy_bindings,p
olicy_templates} \
    dt-iam-terraform/examples/complete
cd dt-iam-terraform
```

Use a quoted heredoc — `<<'EOF'`, not `<<EOF`. These files contain `${...}` and `$$...`; an unquoted heredoc lets your shell expand them and you will write out

broken HCL.

The five modules map 1:1 onto the IAM object model:

Module	Resource
modules/iam_groups	dynatrace_iam_group
modules/iam_policies	dynatrace_iam_policy (account level)
modules/policy_boundaries	dynatrace_iam_policy_boundary
modules/policy_bindings	dynatrace_iam_policy_bindings_v2 (parameters + boundaries)
modules/policy_templates	none — emits verified statement strings the others consume

Shared `versions.tf` — every module needs the same pin, so write it into all five:

```
for m in iam_groups iam_policies policy_boundaries policy_bindings
policy_templates; do
  cat &gt; "modules/$m/versions.tf" &lt;&lt;'EOF'
terraform {
  required_version = "&gt;= 1.3"

  required_providers {
    dynatrace = {
      source  = "dynatrace-oss/dynatrace"
      version = "&gt;= 1.60.0"
    }
  }
}
EOF
done
```

`modules/policy_templates/main.tf` — Ready-to-use policy **statements** (strings only, no resources). Feeds `iam_policies`.

```

cat > modules/policy_templates/main.tf <<'EOF'
# Policy Templates Module
# Ready-to-use policy STATEMENTS (strings only – no resources) aligned with
the
# BPN IAM notebooks (IAM-04 policy authoring, IAM-10 templated assignments,
# IAM-11 persona workshop). Feed these into the iam_policies module.
#
# Statements are additive: multiple ALLOW lines combine. There is no implicit
# access – anything not ALLOWed is denied. Avoid DENY (it overrides ALLows
# granted by other groups).
#
# The team template uses ${bindParam:team} placeholders (written as
# $$${bindParam:team} in HCL to escape interpolation). Resolve them per group
# at binding time via the `parameters` map on the policy binding – one policy,
# many bindings (BPN IAM-10).

# -----
--
# Persona statements
# -----
--
locals {
  # Standard User – read-only visibility (BPN IAM-11)
  standard_user_statement = <<-EOT
    ALLOW environment:roles:viewer;
    ALLOW app-engine:apps:run;
    ALLOW document:documents:read;
    ALLOW storage:metrics:read;
    ALLOW storage:logs:read;
    ALLOW storage:events:read;
    ALLOW storage:entities:read;
    ALLOW settings:objects:read, settings:schemas:read;
  EOT

  # Power User – read everything, write dashboards/notebooks, manage alerting
  # config only (schema-scoped write, BPN IAM-11 GLOBAL-PowerUser-Config)
  power_user_statement = <<-EOT
    ALLOW environment:roles:viewer;
    ALLOW app-engine:apps:run;
    ALLOW document:documents:read, document:documents:write,
document:documents:delete;
    ALLOW storage:metrics:read;
    ALLOW storage:logs:read;
    ALLOW storage:spans:read;
    ALLOW storage:events:read;
    ALLOW storage:bizevents:read;
    ALLOW storage:entities:read;

```

```

    ALLOW storage:system:read;
    ALLOW storage:buckets:read;
    ALLOW automation:workflows:read, automation:workflows:run;
    ALLOW settings:objects:read, settings:schemas:read;
    ALLOW settings:objects:write WHERE settings:schemaId startsWith
"builtin:alerting.profile";
    ALLOW settings:objects:write WHERE settings:schemaId startsWith
"builtin:problem.notifications";
    EOT

# Admin – all data + settings (Gen3) plus Classic roles (Gen2 transitional).
# NOTE: there is no environment:roles:admin permission – Classic admin means
# viewer + manage-settings (BPN IAM-05). Verified against the live Account
# Management API (invalid names are rejected server-side with HTTP 400).
admin_statement = &lt;&lt;-EOT
    ALLOW environment:roles:viewer, environment:roles:manage-settings;
    ALLOW app-engine:apps:run, app-engine:apps:install;
    ALLOW document:documents:read, document:documents:write,
document:documents:delete;
    ALLOW storage:metrics:read;
    ALLOW storage:logs:read, storage:logs:write;
    ALLOW storage:spans:read;
    ALLOW storage:events:read, storage:events:write;
    ALLOW storage:bizevents:read;
    ALLOW storage:entities:read;
    ALLOW storage:system:read;
    ALLOW storage:buckets:read;
    ALLOW automation:workflows:read, automation:workflows:write,
automation:workflows:run;
    ALLOW settings:objects:read, settings:objects:write,
settings:schemas:read;
    EOT

# -----
--
# Parameterized team template (Gen3 – Grail security context)
# One policy for ALL teams; bind per group with parameters = { team = "..."}
# and attach the team's standalone boundary for defense-in-depth (IAM-05).
# -----
--
team_data_access_template = &lt;&lt;-EOT
    ALLOW environment:roles:viewer;
    ALLOW app-engine:apps:run;
    ALLOW document:documents:read;
    ALLOW storage:logs:read WHERE storage:dt.security_context IN
("${bindParam:team}");
    ALLOW storage:spans:read WHERE storage:dt.security_context IN

```

```

("${bindParam:team}");
    ALLOW storage:metrics:read WHERE storage:dt.security_context IN
("${bindParam:team}");
    ALLOW storage:events:read WHERE storage:dt.security_context IN
("${bindParam:team}");
    ALLOW storage:bizevents:read WHERE storage:dt.security_context IN
("${bindParam:team}");
    ALLOW settings:objects:read WHERE settings:dt.security_context IN
("${bindParam:team}");
    ALLOW settings:schemas:read;
EOT

# Gen2 / Classic transitional statement – management-zone scoped write on
# top of global read (MZ2POL pattern; retire together with your MZs).
mz_scoped_config_template = &lt;&lt;-EOT
    ALLOW environment:roles:viewer;
    ALLOW settings:objects:read, settings:schemas:read;
    ALLOW settings:objects:write WHERE environment:management-zone IN
("${bindParam:zone}");
EOT
}

# -----
--
# Boundary query helpers (for the policy_boundaries module)
# -----
--
locals {
    # Gen3 boundary per team – standardize on dt.security_context (IAM-05)
    gen3_team_boundary_query_example = "storage:dt.security_context IN
(\"checkout\"); settings:dt.security_context IN (\"checkout\");"

    # Gen2 boundary – management-zone is the Classic universal scoping field
    gen2_mz_boundary_query_example = "environment:management-zone IN
(\"Checkout\");"
}

output "standard_user_statement" {
    description = "Read-only persona statement"
    value      = local.standard_user_statement
}

output "power_user_statement" {
    description = "Power user persona statement (read all, write documents +
alerting config)"
    value      = local.power_user_statement
}

```

```

output "admin_statement" {
  description = "Admin persona statement"
  value      = local.admin_statement
}

output "team_data_access_template" {
  description = "Parameterized Gen3 team statement – bind with parameters = {
team = \"&lt;security-context&gt;\" }"
  value      = local.team_data_access_template
}

output "mz_scoped_config_template" {
  description = "Parameterized Gen2 transitional statement – bind with
parameters = { zone = \"&lt;mz-name&gt;\" }"
  value      = local.mz_scoped_config_template
}

output "boundary_query_examples" {
  description = "Example boundary query formats for the policy_boundaries
module"
  value = {
    gen3_team = local.gen3_team_boundary_query_example
    gen2_mz   = local.gen2_mz_boundary_query_example
  }
}
EOF

```

`modules/iam_groups/main.tf` — `dynatrace_iam_group` — persona and team groups.


```

cat &gt; modules/iam_groups/main.tf &lt;&lt;'EOF'
# IAM Groups Module
# Manages Dynatrace IAM groups (Account Management API).
#
# Schema verified against dynatrace-oss/dynatrace v1.100.0:
#   dynatrace_iam_group supports: name (required), description,
#   federated_attribute_values (set of strings for SSO/SAML group mapping).
#
# Group membership is managed either via SSO federated attributes
(recommended,
# see IAM-02 in the BPN IAM notebooks) or via dynatrace_iam_user resources
that
# reference these group IDs – NOT on the group resource itself.

variable "groups" {
  description = "Map of IAM group configurations, keyed by a stable logical
name"
  type = map(object({
    name          = string
    description    = optional(string, "Managed by Terraform")

    # SAML/SSO federated attribute values that map IdP groups to this group.
    # Leave empty when membership is managed manually or via
dynatrace_iam_user.
    federated_attribute_values = optional(set(string), [])
  }))
  default = {}
}

resource "dynatrace_iam_group" "this" {
  for_each = var.groups

  name          = each.value.name
  description    = each.value.description
  federated_attribute_values = length(each.value.federated_attribute_values)
&gt; 0 ? each.value.federated_attribute_values : null
}

output "group_ids" {
  description = "Map of logical group keys to group UUIDs (use in policy
bindings)"
  value = {
    for group_key, group in dynatrace_iam_group.this :
    group_key => group.id
  }
}

```

```
output "groups" {
  description = "Map of created IAM groups with details"
  value = {
    for group_key, group in dynatrace_iam_group.this :
    group_key => {
      id      = group.id
      name    = group.name
      description = group.description
    }
  }
}
EOF
```

`modules/iam_policies/main.tf` — `dynatrace_iam_policy` at **account** level.

```

cat &gt; modules/iam_policies/main.tf &lt;&lt;'EOF'
# IAM Policies Module
# Manages Dynatrace IAM policies using official permission syntax.
# Reference: https://docs.dynatrace.com/docs/manage/identity-access-management/permission-management
#
# Schema verified against dynatrace-oss/dynatrace v1.100.0:
#   dynatrace_iam_policy supports: name (required), statement_query
#   (required),
#   account XOR environment (scope), description, tags (set of strings).
#
# Best practice (BPN IAM-04 / MZ2POL-04): create policies at ACCOUNT level and
# scope them per group at binding time via ${bindParam:...} parameters and
# boundaries – do NOT create per-environment policy copies. The `environment`
# attribute is deprecated in the provider for exactly this reason.
#
# Statement syntax:
#   ALLOW &lt;service&gt;;&lt;resource&gt;;&lt;action&gt;;
#   ALLOW storage:logs:read WHERE storage:dt.security_context = "checkout";
#   ALLOW storage:logs:read WHERE storage:dt.security_context =
#   "${bindParam:team}";
# (in HCL heredocs write $$${bindParam:team} to escape Terraform interpolation)

variable "account_id" {
  description = "Dynatrace account UUID – policies are created at account
level"
  type        = string
}

variable "policies" {
  description = "Map of IAM policy configurations, keyed by a stable logical
name"
  type = map(object({
    name          = string
    description    = optional(string, "Managed by Terraform")

    # Policy statement using Dynatrace permission syntax (may contain
    # $$${bindParam:...} placeholders resolved per binding).
    statement      = string

    tags = optional(set(string), [])
  }))
  default = {}
}

resource "dynatrace_iam_policy" "this" {
  for_each = var.policies

```

```

    name          = each.value.name
    description    = each.value.description
    account        = var.account_id
    statement_query = each.value.statement
    tags           = length(each.value.tags) > 0 ? each.value.tags : null
  }

  output "policy_ids" {
    description = "Map of logical policy keys to policy IDs (accepted by policy
bindings)"
    value = {
      for policy_key, policy in dynatrace_iam_policy.this :
        policy_key => policy.id
    }
  }

  output "policy_uuids" {
    description = "Map of logical policy keys to bare policy UUIDs"
    value = {
      for policy_key, policy in dynatrace_iam_policy.this :
        policy_key => policy.uuid
    }
  }

  output "policies" {
    description = "Map of created policies with details"
    value = {
      for policy_key, policy in dynatrace_iam_policy.this :
        policy_key => {
          id          = policy.id
          uuid        = policy.uuid
          name        = policy.name
          description = policy.description
        }
    }
  }
}
EOF

```

`modules/policy_boundaries/main.tf` — `dynatrace_iam_policy_boundary` — standalone, account-level.

```

cat &gt; modules/policy_boundaries/main.tf &lt;&lt;'EOF'
# Policy Boundaries Module
# Manages Dynatrace IAM policy boundaries – standalone, account-level IAM
# objects that restrict WHAT DATA a policy binding can reach.
#
# Schema verified against dynatrace-oss/dynatrace v1.100.0:
#   dynatrace_iam_policy_boundary supports ONLY: name (required), query
#   (required).
#   (No description, no environment – boundaries are account-level objects and
#   are attached to groups via the `boundaries` list on policy binding
#   blocks.)
#
# Boundary query syntax (BPN IAM-05):
#   Gen3 (Grail/AppEngine policies):
#     storage:dt.security_context IN ("checkout");
#     settings:dt.security_context IN ("checkout");
#     storage:bucket-name IN ("checkout_logs");
#   Gen2 / Classic (environment:* policies, transitional while MZs still
#   exist):
#     environment:management-zone IN ("Checkout");
#
# Pair Gen2 boundaries with Gen2 policies and Gen3 boundaries with Gen3
# policies – conditions from the wrong generation are silently ignored.
# Best practice: one boundary per team with hardcoded scope (the policy stays
# parameterized; the boundary is the defense-in-depth layer). Max 10
# conditions
# per boundary – create multiple boundaries if you need more.

variable "boundaries" {
  description = "Map of policy boundary configurations, keyed by a stable
  logical name"
  type = map(object({
    name = string

    # Boundary query, e.g.:
    #   storage:dt.security_context IN ("checkout");
    settings:dt.security_context IN ("checkout");
    query = string
  }))
  default = {}
}

resource "dynatrace_iam_policy_boundary" "this" {
  for_each = var.boundaries

  name   = each.value.name
  query = each.value.query

```

```

}

output "boundary_ids" {
  description = "Map of logical boundary keys to boundary UUIDs (attach via
policy binding `boundaries`)"
  value = {
    for boundary_key, boundary in dynatrace_iam_policy_boundary.this :
      boundary_key => boundary.id
  }
}

output "boundaries" {
  description = "Map of created boundaries with details"
  value = {
    for boundary_key, boundary in dynatrace_iam_policy_boundary.this :
      boundary_key => {
        id      = boundary.id
        name    = boundary.name
        query   = boundary.query
      }
  }
}
EOF

```

`modules/policy_bindings/main.tf` — `dynatrace_iam_policy_bindings_v2` — where groups, policies, parameters and boundaries meet.

```

cat &gt; modules/policy_bindings/main.tf &lt;&lt;&lt;'EOF'
# Policy Bindings Module
# Connects IAM groups to policies via dynatrace_iam_policy_bindings_v2.
#
# Schema verified against dynatrace-oss/dynatrace v1.100.0:
#   dynatrace_iam_policy_bindings_v2 supports: group (required),
#   account XOR environment (scope), and repeated `policy` blocks with:
#     id          - id (or uuid) of a dynatrace_iam_policy
#     parameters - map(string) resolving ${bindParam:...} placeholders
#     boundaries  - set of dynatrace_iam_policy_boundary IDs
#     metadata    - map(string) free-form metadata
#
# One binding resource per (group, scope) pair - it OWNS the full set of
# policies bound to that group at that scope. This is where the BPN
# "templated policy" pattern lands: one parameterized policy, many bindings
# with different `parameters` and per-team `boundaries` (IAM-10, IAM-05).

variable "bindings" {
  description = "Map of policy binding configurations, keyed by a stable
logical name. Exactly one of account_id / environment_id must be set per
binding."
  type = map(object({
    group_id = string

    # Scope: account UUID for account-level bindings (policies must be
    # account-level), or environment ID (e.g. `abc12345`) for
    # environment-level bindings.
    account_id   = optional(string)
    environment_id = optional(string)

    policies = list(object({
      id          = string
      parameters = optional(map(string), {})
      boundaries  = optional(set(string), [])
      metadata    = optional(map(string), {})
    }))
  }))
  default = {}

  validation {
    condition = alltrue([
      for binding in values(var.bindings) :
      (binding.account_id != null) != (binding.environment_id != null)
    ])
    error_message = "Each binding must set exactly one of account_id or
environment_id."
  }
}

```

```

}

resource "dynatrace_iam_policy_bindings_v2" "this" {
  for_each = var.bindings

  group      = each.value.group_id
  account    = each.value.account_id
  environment = each.value.environment_id

  dynamic "policy" {
    for_each = each.value.policies
    content {
      id          = policy.value.id
      parameters = length(policy.value.parameters) > 0 ?
policy.value.parameters : null
      boundaries = length(policy.value.boundaries) > 0 ?
policy.value.boundaries : null
      metadata   = length(policy.value.metadata) > 0 ?
policy.value.metadata : null
    }
  }
}

output "bindings" {
  description = "Map of created bindings with details"
  value = {
    for binding_key, binding in dynatrace_iam_policy_bindings_v2.this :
binding_key => {
      id          = binding.id
      group       = binding.group
      account     = binding.account
      environment = binding.environment
    }
  }
}
EOF

```

`examples/complete/variables.tf` — Inputs for the composition.


```

cat &gt; examples/complete/variables.tf &lt;&lt;'EOF'
variable "dynatrace_account_id" {
    description = "Dynatrace account UUID (Account Management &gt; Account
settings)"
    type        = string
}

variable "dynatrace_client_id" {
    description = "OAuth client ID for the Account Management API (dt0s02....)"
    type        = string
    sensitive   = true
}

variable "dynatrace_client_secret" {
    description = "OAuth client secret for the Account Management API"
    type        = string
    sensitive   = true
}

variable "teams" {
    description = &lt;&lt;-EOT
        Teams to onboard. Each team gets a group, a standalone boundary, and one
        binding of the shared parameterized policy. `security_context` must match
        the dt.security_context value stamped on the team's data (Gen3).
        `management_zone` (optional) adds a Gen2/Classic condition to the boundary
        while management zones still exist (MZ2POL transitional pattern).
    EOT
    type = map(object({
        name                = string
        security_context     = string
        management_zone      = optional(string)
        federated_attribute_values = optional(set(string), [])
    }))
    default = {}
}

variable "name_prefix" {
    description = "Optional prefix for every group/policy/boundary name – useful
for test runs and multi-instance deployments"
    type        = string
    default     = ""
}

variable "binding_environment_id" {
    description = "If set (e.g. \"abc12345\"), all policy bindings attach at
ENVIRONMENT level to this environment instead of account level – scopes
granted access to a single environment"

```

```
    type      = string
    default   = null
  }
EOF
```

`examples/complete/main.tf` — The composition: personas + one parameterized policy shared by every team.

```

cat &gt; examples/complete/main.tf &lt;&lt;'EOF'
# Complete example: Dynatrace IAM with groups, policies, boundaries and
bindings
#
# Implements the BPN best-practice pattern (IAM-04/05/10/11, MZ2POL-04):
#
# 1. Persona groups (Standard / Power / Admin) bound to persona policies
#    at ACCOUNT level – one policy each, no duplication.
# 2. Team groups driven entirely by `var.teams`:
#    - ONE parameterized policy (`${bindParam:team}` on
dt.security_context)
#    shared by every team,
#    - one standalone boundary per team (defense-in-depth),
#    - one binding per team supplying parameters + boundary.
#    Adding a team = adding one entry to terraform.tfvars. No new policies.
#
# Everything here validates against dynatrace-oss/dynatrace v1.100.0.

terraform {
  required_version = "&gt;= 1.3"

  required_providers {
    dynatrace = {
      source = "dynatrace-oss/dynatrace"
      version = "&gt;= 1.60.0"
    }
  }
}

provider "dynatrace" {
  # IAM resources authenticate via an Account Management OAuth client.
  iam_account_id    = var.dynatrace_account_id
  iam_client_id     = var.dynatrace_client_id
  iam_client_secret = var.dynatrace_client_secret
}

#
=====
# Policy statement templates (strings only – verified BPN syntax)
#
=====
module "policy_templates" {
  source = "../modules/policy_templates"
}

#
=====

```

```

# Groups: three personas + one group per team
#
=====
module "iam_groups" {
    source = "../../modules/iam_groups"

    groups = merge(
        {
            standard_users = {
                name          = "${var.name_prefix}Standard Users"
                description = "Read-only access to dashboards, metrics, logs and
events"
            }
            power_users = {
                name          = "${var.name_prefix}Power Users"
                description = "Read everything; write dashboards/notebooks and
alerting config"
            }
            admins = {
                name          = "${var.name_prefix}Administrators"
                description = "Full environment control"
            }
        },
        {
            for team_key, team in var.teams :
                "team_${team_key}" => {
                    name          = "${var.name_prefix}${team.name} Team"
                    description = "Team-scoped access to the
'${team.security_context}' security context"
                    federated_attribute_values = team.federated_attribute_values
                }
        }
    )
}

#
=====
# Policies: one per persona + ONE parameterized policy shared by all teams
#
=====
module "iam_policies" {
    source = "../../modules/iam_policies"

    account_id = var.dynatrace_account_id

    policies = {
        standard_user = {
            name          = "${var.name_prefix}base-standard-user"

```

```

        description = "Persona: read-only visibility"
        statement   = module.policy_templates.standard_user_statement
        tags        = ["persona", "managed-by-terraform"]
    }

    power_user = {
        name         = "${var.name_prefix}base-power-user"
        description   = "Persona: read all, write documents and alerting config"
        statement     = module.policy_templates.power_user_statement
        tags          = ["persona", "managed-by-terraform"]
    }

    admin = {
        name          = "${var.name_prefix}base-admin"
        description   = "Persona: full environment control"
        statement     = module.policy_templates.admin_statement
        tags          = ["persona", "managed-by-terraform"]
    }

    # One template for ALL teams – resolved per binding via parameters.team
    team_data_access = {
        name          = "${var.name_prefix}tpl-team-data-access"
        description   = "Parameterized: data access scoped to ${bindParam:team}
security context"
        statement     = module.policy_templates.team_data_access_template
        tags          = ["template", "managed-by-terraform"]
    }
}

#
=====
# Boundaries: one standalone boundary per team (hardcoded scope is intentional
# – the policy carries the parameterized scope, the boundary is the
# defense-in-depth layer; BPN IAM-05)
#
=====
module "policy_boundaries" {
    source = "../../modules/policy_boundaries"

    boundaries = {
        for team_key, team in var.teams :
        "team_${team_key}" => {
            name = "${var.name_prefix}bnd-team-${team_key}"
            query = join(" ", concat(
                [
                    "storage:dt.security_context IN (\`${team.security_context}\`);",
                    "settings:dt.security_context IN (\`${team.security_context}\`);",

```

```

    ],
    # Gen2/Classic condition – transitional while management zones exist
    team.management_zone != null ? ["environment:management-zone IN
(\"${team.management_zone}\");"] : []
  ))
}
}
}

#
=====
# Bindings: personas + one parameterized binding per team.
# Bound at account level by default; set binding_environment_id to scope all
# granted access to a single environment instead.
#
=====
locals {
  bind_account_id = var.binding_environment_id == null ?
var.dynatrace_account_id : null
}

module "policy_bindings" {
  source = "../../modules/policy_bindings"

  bindings = merge(
    {
      standard_users = {
        group_id      = module.iam_groups.group_ids["standard_users"]
        account_id    = local.bind_account_id
        environment_id = var.binding_environment_id
        policies      = [{ id =
module.iam_policies.policy_ids["standard_user"] }]
      }
      power_users = {
        group_id      = module.iam_groups.group_ids["power_users"]
        account_id    = local.bind_account_id
        environment_id = var.binding_environment_id
        policies      = [{ id = module.iam_policies.policy_ids["power_user"]
}]
    }
    admins = {
      group_id      = module.iam_groups.group_ids["admins"]
      account_id    = local.bind_account_id
      environment_id = var.binding_environment_id
      policies      = [{ id = module.iam_policies.policy_ids["admin"] }]
    }
  },
  {

```

```

    for team_key, team in var.teams :
    "team_${team_key}" => {
        group_id      = module.iam_groups.group_ids["team_${team_key}"]
        account_id    = local.bind_account_id
        environment_id = var.binding_environment_id
        policies = [{
            id          = module.iam_policies.policy_ids["team_data_access"]
            parameters = { team = team.security_context }
            boundaries =
[module.policy_boundaries.boundary_ids["team_${team_key}"]]
            metadata    = { managed-by = "terraform" }
        }]
    }
}
)
}

#
=====
# Outputs
#
=====
output "groups" {
    description = "Created IAM groups"
    value       = module.iam_groups.groups
}

output "policies" {
    description = "Created IAM policies"
    value       = module.iam_policies.policies
}

output "boundaries" {
    description = "Created policy boundaries"
    value       = module.policy_boundaries.boundaries
}

output "bindings" {
    description = "Created policy bindings"
    value       = module.policy_bindings.bindings
}
EOF

```

examples/complete/terraform.tfvars.example — Template for your values. Copy to **terraform.tfvars** in Step 2.

```

cat > examples/complete/terraform.tfvars.example <<<'EOF'
# Rename to terraform.tfvars and fill in your values.
# NEVER commit terraform.tfvars – it contains credentials.
#
# The OAuth client is created in Account Management > Identity & access
# management > OAuth clients. Required scopes:
#   account-idm-read account-idm-write
#   iam:policies:read iam:policies:write
#   iam:bindings:read iam:bindings:write
#   iam:boundaries:read iam:boundaries:write

dynatrace_account_id    = "00000000-0000-0000-0000-000000000000"
dynatrace_client_id     = "dt0s02.XXXXXXXX"
dynatrace_client_secret = "dt0s02.XXXXXXXX.YYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYY"

# Adding a team here is ALL that is needed to onboard it:
teams = {
  checkout = {
    name           = "Checkout"
    security_context = "checkout"
    management_zone = "Checkout" # optional – drop once MZs are retired
  }

  platform = {
    name           = "Platform"
    security_context = "platform"
  }
}
EOF

```

That is the whole project. Sanity-check the layout before moving on:

```
find . -name '*.tf' -o -name '*.tfvars.example' | sort
```

You should see 12 `.tf` files (five modules x `main.tf` + `versions.tf`, plus the example's `main.tf` and `variables.tf`) and one `terraform.tfvars.example`.

Step 2 — Configure Credentials and Teams

```
cp terraform.tfvars.example terraform.tfvars
```

Edit `terraform.tfvars`:


```

dynatrace_account_id    = "&lt;your-account-uuid&gt;"
dynatrace_client_id     = "dt0s02.XXXXXXXX"
dynatrace_client_secret = "dt0s02.XXXXXXXX.YYYY..."

name_prefix             = "TFTEST-"           # every object gets this prefix
binding_environment_id = "abc12345"           # YOUR test environment ID

teams = {
  alpha = {
    name              = "Alpha"
    security_context = "tfctest-alpha"
    management_zone   = "TFTEST-Alpha"      # optional Gen2 condition (MZ2POL
transitional)
  }
  beta = {
    name              = "Beta"
    security_context = "tfctest-beta"      # no MZ -&gt; pure Gen3 boundary
  }
}

```

`terraform.tfvars` contains credentials — it is gitignored. Never commit it.

Step 3 — Initialize and Plan

```

terraform init
terraform plan

```

Expected output (tail of plan):

```

Plan: 16 to add, 0 to change, 0 to destroy.

```

16 resources = 5 groups (3 personas + 2 teams) + 4 policies (3 personas + 1 shared template) + 2 boundaries + 5 bindings. Inspect the parameterized policy in the plan — the `${bindParam:team}` placeholders must appear **literally** (they are resolved per binding, not by Terraform):

```
+ statement_query = &lt;&lt;-EOT
    ALLOW environment:roles:viewer;
    ALLOW app-engine:apps:run;
    ALLOW document:documents:read;
    ALLOW storage:logs:read WHERE storage:dt.security_context IN
("${bindParam:team}");
...

```

Step 4 — Apply

```
terraform apply
```

Type `yes` at the prompt. **Expected output (abridged):**

```
module.iam_groups.dynatrace_iam_group.this["team_alpha"]: Creation complete
after 0s [id=00d43484-fc8a-435f-8e4f-57ea9d056d35]
module.policy_boundaries.dynatrace_iam_policy_boundary.this["team_alpha"]:
Creation complete after 0s [id=d81e7853-abd4-4159-908f-75c202432cf1]
module.iam_policies.dynatrace_iam_policy.this["team_data_access"]: Creation
complete after 1s [id=dea9e886-...#-#account#-#&lt;account-uuid&gt;]
module.policy_bindings.dynatrace_iam_policy_bindings_v2.this["team_alpha"]:
Creation complete after 1s [id=00d43484-...#-#environment#-#abc12345]

Apply complete! Resources: 16 added, 0 changed, 0 destroyed.
```

Note the binding IDs end in `#-#environment#-#abc12345` — proof the grants are environment-scoped.

Step 5 — Validate

In the UI — Account Management → Identity & access management:

- **Groups:** search `TFTEST-` → 5 groups
- **Policies:** open `TFTEST-tpl-team-data-access` → statement shows `${bindParam:team}` placeholders; its **bindings** tab shows per-team resolved values
- **Boundaries:** `TFTEST-bnd-team-alpha` shows all three conditions; `TFTEST-bnd-team-beta` shows Gen3 conditions only

Via the API (or use the IAM-96 Python tool). **Expected binding for the Alpha team:**

```
{
  "policyUuid": "dea9e886-d49d-4989-ac15-c193e823dd00",
  "groups": ["00d43484-fc8a-435f-8e4f-57ea9d056d35"],
  "parameters": { "team": "tftest-alpha" },
  "boundaries": ["d81e7853-abd4-4159-908f-75c202432cf1"]
}
```

Expected boundary query (Alpha — with transitional MZ condition):

```
storage:dt.security_context IN ("tftest-alpha");
settings:dt.security_context IN ("tftest-alpha");
environment:management-zone IN ("TFTEST-Alpha");
```

✓ **Checkpoint:** `parameters.team` matches the team's `security_context`, and exactly one boundary UUID is attached.

Step 6 — The Payoff: Onboard a Team with Three Lines

Add to `teams` in `terraform.tfvars`:

```
gamma = {
  name           = "Gamma"
  security_context = "tftest-gamma"
}
```

```
terraform plan
```

Expected output:

```
Plan: 3 to add, 0 to change, 0 to destroy.
```

Three resources — group, boundary, binding. **The shared policy is untouched.** That is the entire value of the templated-policy pattern: teams scale without policy sprawl. Apply if you want, then continue.

Step 7 — Clean Up

```
terraform destroy
```

Expected output (tail):

```
Destroy complete! Resources: 16 destroyed.
```

(19 if you applied Gamma.) Verify in the UI that no `TFTEST-` objects remain, then delete `terraform.tfvars`.

Troubleshooting

Symptom	Cause	Fix
400 Invalid permission name: ... on apply	Statement uses a nonexistent permission (e.g. <code>environment:roles:admin</code> , <code>storage:spans:write</code>)	See the verified permission rules in IAM-04 and error table in IAM-12 §7
400 ALLOW or DENY must be followed by a permission...	Wildcard in a statement (<code>storage*:read</code>)	Wildcards are not supported — enumerate permissions
401 on plan/apply	OAuth client credentials wrong or scopes missing	Recreate client with the full scope list above
Binding created at account level unexpectedly	<code>binding_environment_id</code> unset	Set it — null means account-level bindings
<code>\${bindParam:team}</code> resolved to nothing by Terraform	Written as <code>\${...}</code> in HCL	In HCL heredocs escape as <code>`\${bindParam:team}</code>

Validation Checklist

- `[] terraform plan` showed 16 to add before apply

- [] Binding IDs contain `##environment##<env-id>`
- [] The shared policy statement contains literal `${bindParam:team}`
- [] Alpha's binding carries `parameters: {team: tftest-alpha}` + 1 boundary
- [] Beta's boundary has NO `environment:management-zone` line (pure Gen3)
- [] Adding a team planned exactly **3** new resources
- [] `terraform destroy` removed everything (UI search for prefix is empty)

References

- **IAM-04** Policy Authoring (verified statement syntax) · **IAM-05** Boundary Design · **IAM-10** Templated Policy Assignments · **IAM-12** API Provisioning (curl flavor) · **IAM-96** Python lab (same pattern, Python flavor)
 - **MZ2POL series** — migrating off Management Zones onto this pattern
 - **AUTOM-95 LAB** (AUTOM series) — provider-lifecycle depth for the same `dynatrace_iam_*` resources: permission-DSL discovery, bulk export/import of an existing account, deprecated arguments, CI/CD integration
 - Provider reference for every resource used here: `dynatrace_iam_group` , `dynatrace_iam_policy` , `dynatrace_iam_policy_boundary` , `dynatrace_iam_policy_bindings_v2` — the HCL in Step 1 is schema-verified against provider v1.100.0
-

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.